```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
breast = pd.read_csv("breast cancer.csv")
```

```
breast.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	con points_
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.1
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.1
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.1

5 rows × 33 columns

```
breast.shape
```

```
(569, 33)
```

```
breast.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

2. Data Preprocessing

```
breast.isnull().sum()
```

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0
fractal_dimension_worst	0
Unnamed: 32	569

dtype: int64

```
breast.drop('Unnamed: 32', axis=1, inplace=True)
```

```
breast.duplicated().sum()
```

```
np.int64(0)
```

```
breast.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concav points_mea
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.0000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.0489
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.0388
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.0000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.0203
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.0335
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.0740
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.2012

8 rows × 31 columns

Encoding the Target Variable

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()


breast['diagnosis'] = le.fit_transform(breast['diagnosis'])


#breast['diagnosis'] =breast['diagnosis'].map({"M":1,"B":0})
```

Splitting Data into Training and Testing Sets

```
from sklearn.model_selection import train_test_split
# Split the data into training and testing sets
X = breast.drop('diagnosis', axis=1)
y = breast['diagnosis']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape)
print(X_test.shape)
```

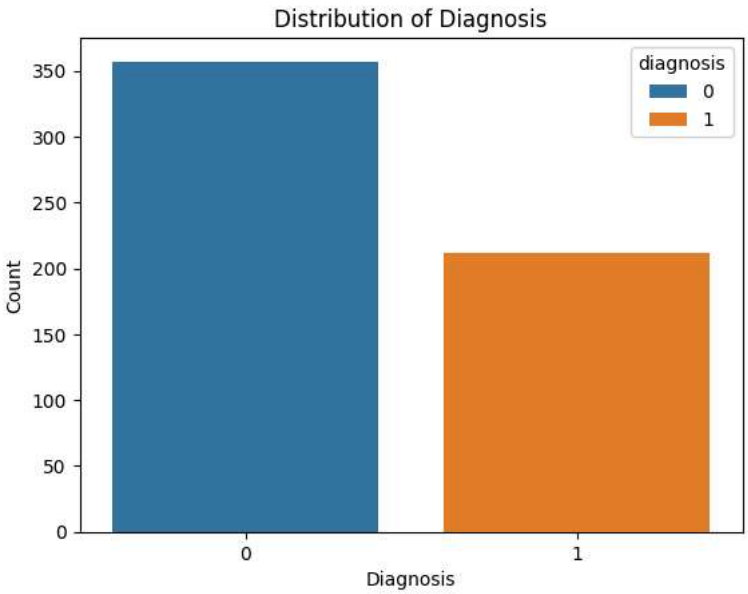
```
(455, 31)
(114, 31)
```

Feature Scaling

```
from sklearn.preprocessing import StandardScaler
# Scale the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Exploratory Data Analysis (EDA)

```
# Visualize the distribution of the target variable
sns.countplot(x=breast['diagnosis'], hue=breast['diagnosis'])
plt.title('Distribution of Diagnosis')
plt.xlabel('Diagnosis')
plt.ylabel('Count')
plt.show()
```




```
# Visualize the correlation matrix
breast.corr()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	conc
id	1.000000	0.039769	0.074626	0.099770	0.073159	0.096893	-0.012968	0.000096	
diagnosis	0.039769	1.000000	0.730029	0.415185	0.742636	0.708984	0.358560	0.596534	
radius_mean	0.074626	0.730029	1.000000	0.323782	0.997855	0.987357	0.170581	0.506124	
texture_mean	0.099770	0.415185	0.323782	1.000000	0.329533	0.321086	-0.023389	0.236702	
perimeter_mean	0.073159	0.742636	0.997855	0.329533	1.000000	0.986507	0.207278	0.556936	
area_mean	0.096893	0.708984	0.987357	0.321086	0.986507	1.000000	0.177028	0.498502	
smoothness_mean	-0.012968	0.358560	0.170581	-0.023389	0.207278	0.177028	1.000000	0.659123	
compactness_mean	0.000096	0.596534	0.506124	0.236702	0.556936	0.498502	0.659123	1.000000	
concavity_mean	0.050080	0.696360	0.676764	0.302418	0.716136	0.685983	0.521984	0.883121	
concave points_mean	0.044158	0.776614	0.822529	0.293464	0.850977	0.823269	0.553695	0.831135	
symmetry_mean	-0.022114	0.330499	0.147741	0.071401	0.183027	0.151293	0.557775	0.602641	
fractal_dimension_mean	-0.052511	-0.012838	-0.311631	-0.076437	-0.261477	-0.283110	0.584792	0.565369	
radius_se	0.143048	0.567134	0.679090	0.275869	0.691765	0.732562	0.301467	0.497473	
texture_se	-0.007526	-0.008303	-0.097317	0.386358	-0.086761	-0.066280	0.068406	0.046205	
perimeter_se	0.137331	0.556141	0.674172	0.281673	0.693135	0.726628	0.296092	0.548905	
area_se	0.177742	0.548236	0.735864	0.259845	0.744983	0.800086	0.246552	0.455653	
smoothness_se	0.096781	-0.067016	-0.222600	0.006614	-0.202694	-0.166777	0.332375	0.135299	
compactness_se	0.033961	0.292999	0.206000	0.191975	0.250744	0.212583	0.318943	0.738722	
concavity_se	0.055239	0.253730	0.194204	0.143293	0.228082	0.207660	0.248396	0.570517	
concave points_se	0.078768	0.408042	0.376169	0.163851	0.407217	0.372320	0.380676	0.642262	
symmetry_se	-0.017306	-0.006522	-0.104321	0.009127	-0.081629	-0.072497	0.200774	0.229977	
fractal_dimension_se	0.025725	0.077972	-0.042641	0.054458	-0.005523	-0.019887	0.283607	0.507318	
radius_worst	0.082405	0.776454	0.969539	0.352573	0.969476	0.962746	0.213120	0.535315	
texture_worst	0.064720	0.456903	0.297008	0.912045	0.303038	0.287489	0.036072	0.248133	
perimeter_worst	0.079986	0.782914	0.965137	0.358040	0.970387	0.959120	0.238853	0.590210	
area_worst	0.107187	0.733825	0.941082	0.343546	0.941550	0.959213	0.206718	0.509604	
smoothness_worst	0.010338	0.421465	0.119616	0.077503	0.150549	0.123523	0.805324	0.565541	
compactness_worst	-0.002968	0.590998	0.413463	0.277830	0.455774	0.390410	0.472468	0.865809	
concavity_worst	0.023203	0.659610	0.526911	0.301025	0.563879	0.512606	0.434926	0.816275	
concave points_worst	0.035174	0.793566	0.744214	0.295316	0.771241	0.722017	0.503053	0.815573	
symmetry_worst	-0.044224	0.416294	0.163953	0.105008	0.189115	0.143570	0.394309	0.510223	
fractal_dimension_worst	-0.029866	0.323872	0.007066	0.119205	0.051019	0.003738	0.499316	0.687382	

32 rows × 32 columns

Training Model

```
# for now we have used logistic regression (you can try more models)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Train the logistic regression model
lr = LogisticRegression()
lr.fit(X_train, y_train)

# Evaluate the performance on the testing set
y_pred_lr = lr.predict(X_test)
accuracy_lr = accuracy_score(y_test, y_pred_lr)
print('Accuracy:', accuracy_lr)
```

```
Accuracy: 0.956140350877193
```

```
# -0.23717126, -0.64487029, -0.11382239, -0.57427777, -0.60294971,
#         1.0897546 ,  0.91543814,  0.41448279,  0.09311633,  1.78465117,
#         2.11520208,  0.28454765, -0.31910982,  0.2980991 ,  0.01968238,
#        -0.47096352,  0.45757106,  0.28733283, -0.23125455,  0.26417944,
#         0.66325388,  0.12170193,  0.42656325,  0.36885508,  0.02065602,
#         1.39513782,  2.0973271 ,  2.01276347,  0.61938913,  2.9421769 ,
#         3.15970842
```

```
#   -0.23711093, -0.4976419 ,  0.61365274, -0.49813131, -0.53102815,
#        -0.57694824, -0.17494424, -0.36215622, -0.284859  ,  0.43345165,
#         0.17818232, -0.36844966,  0.55310406, -0.31671104, -0.40524636,
#         0.04025752, -0.03795529, -0.18043065,  0.16478901, -0.12170969,
#         0.23079329, -0.50044002,  0.81940367, -0.46922838, -0.53308833,
#        -0.04910117, -0.04160193, -0.14913653,  0.09681787,  0.10617647,
#         0.49035329
```

Prediction System

```
import numpy as np

input_text = (-0.23717126, -0.64487029, -0.11382239, -0.57427777, -0.60294971,
        1.0897546 ,  0.91543814,  0.41448279,  0.09311633,  1.78465117,
        2.11520208,  0.28454765, -0.31910982,  0.2980991 ,  0.01968238,
       -0.47096352,  0.45757106,  0.28733283, -0.23125455,  0.26417944,
        0.66325388,  0.12170193,  0.42656325,  0.36885508,  0.02065602,
        1.39513782,  2.0973271 ,  2.01276347,  0.61938913,  2.9421769 ,
        3.15970842)
np_df = np.asarray(input_text)
pred = lr.predict(np_df.reshape(1,-1))
if pred[0] == 1:
    print("Cancrous")
else:
    print("Not Cancrous")
```

```
Cancrous
```

Save Model

```
import pickle
pickle.dump(lr,open('model.pkl','wb'))
```

```
import sklearn

print(sklearn.__version__)  # use the same version in deployment as well (pycharm)
```

```
1.5.1
```

```
Start coding or generate with AI.
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.model_selection import GridSearchCV

# Define the hyperparameter grid for Logistic Regression
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100],
              'penalty': ['l1', 'l2']}

# Create a GridSearchCV object
```

```
grid_search = GridSearchCV(LogisticRegression(solver='liblinear'), param_grid, cv=5)

# Perform hyperparameter tuning on the training data
grid_search.fit(X_train, y_train)

# Print the best hyperparameters and the corresponding accuracy
print("Best hyperparameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.best_score_)

# Evaluate the best model on the testing data
best_lr_model = grid_search.best_estimator_
y_pred_tuned_lr = best_lr_model.predict(X_test)
accuracy_tuned_lr = accuracy_score(y_test, y_pred_tuned_lr)
print("Accuracy on the testing set with tuned hyperparameters:", accuracy_tuned_lr)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear failed to converge, increase the number
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear failed to converge, increase the number
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:1249: ConvergenceWarning: Liblinear failed to converge, increase the number
```